

Overview

This document specifies the **backend-only** work needed to complete three Fill-In features. The mobile app already sends and handles everything described here — no further frontend changes are required. Each section states the endpoint, which app uses it (recruiter / candidate), the screen it powers, why it exists, and the exact request/response shape.

Base URL	https://fillin.mobilesapplication.com/api
Auth	Authorization: Bearer <token> on every endpoint
Envelope	Keep the existing { status: true, message: '...', data: ... } wrapper
Scope	3 features · 4 endpoints · all backend-side

0 · Shared — Notification record schema

Every notification shown in the app comes from **GET /{role}/notification-list**. New notifications (Section 1) must use these same fields so they render and route correctly.

Field	Type	Used for
id	int	Mark-as-read key
type	string	Deep-link routing (the app routes on this)
title	string	Bold line in the bell list
message	string	Body line
icone	string (url)	Left avatar / clinic logo
unique_id	int	Target entity id (shift / request / candidate)
candidate_id	int (optional)	Deep-link to a candidate
read_at	datetime null	null = unread (highlighted styling)
created_at	datetime	Date grouping in the list

Unread badge count comes from **unreadNotification** on the profile endpoints (recruiter-profile / candidate profile) — increment it when a notification is created. Mark-as-read already exists: GET /{role}/notifications/mark-as-read/{id}.

1 · Cancellation notifications (both sides)

The cancellation CRUD is already live. What is missing is a **side-effect** inside two existing endpoints: create a notification record (and optionally an FCM push). **No new endpoints.**

1A · Candidate submits → notify the RECRUITER

Hook into	POST /candidate/request-cancellation (already exists)
Recipient	The recruiter who owns the shift
App / screen	Recruiter bell list → tap → Cancellation Requests (Approve / Reject inline)
Why	Alert the recruiter so they can act immediately

After saving the cancellation request, create:

```
{
  "user_id":      "<shift.recruiter_id>",      // recipient = recruiter
  "type":         "cancellation_request",
  "title":        "Cancellation request",
  "message":      "<candidate_name> requested to cancel <shift_title>",
  "unique_id":    "<cancellation_request_id>",
  "candidate_id": "<candidate_id>",
```

Fill-In · Backend API Specification

Pending backend items — cancellation notifications, fill-in branch, software search

```
"icone":      "<candidate_profile_url>",
"read_at":    null
}
```

1B · Recruiter approves / rejects → notify the CANDIDATE

Hook into	POST /recruiter/update-cancellation-status (already exists)
Recipient	The candidate who made the request
App / screen	Candidate bell list → tap → Cancellation Requests (status + remark)
Why	Candidate learns the outcome of their request

After updating the status, create:

```
{
  "user_id":  "<request.candidate_id>",          // recipient = candidate
  "type":     "cancellation_approved",          // or "cancellation_rejected"
  "title":    "Cancellation approved",          // or "Cancellation rejected"
  "message":  "Your cancellation for <shift_title> was approved",
  "unique_id": "<cancellation_request_id>",
  "icone":    "<clinic_logo_url>",
  "read_at":  null
}
```

Routing rule (already coded in the app): any notification whose type / title / message contains the substring **"cancel"** opens the cancellation screen — so exact strings are flexible, just keep "cancel" in the type.

FCM push (real-time): also push to the recipient's stored fcm_token with the same type in the data payload:

```
{
  "notification": { "title": "...", "body": "...", },
  "data":         { "type": "cancellation_request", "unique_id": "<id>" }
}
```

2 · Fill-In shift branch (branch_ids)

Endpoint	POST /recruiter/search-candidates (already exists — creates the fill-in shift + returns matching candidates)
Side	Recruiter
App / screen	'Find Fill-In Staff' form (FillInShiftForm) → candidate list (FillInCandidateList)
Why	A fill-in shift must be taggable to branches, exactly like create-job already is

Request body the app now sends (existing fields + the new one):

```
{
  "title":      "Dental Assistant Required",
  "specialization_id": 3,
  "experiance_level": "1-3 Years",
  "software":    [1, 2],
  "other_software": "Demo",
  "branch_ids":  [5, 6, 7],          // <== ADD: persist on the shift
  "vacancy":     2,
  "urgent":      1,
  "city":        "New York",
  "address":     "350 5th Avenue, New York",
  "short_address": "New York",
  "job_description": "...",
  "expire_date": "2026-07-31",
  "latitude":    40.7127753,
  "longitude":   -74.0059728,
  "hourly_rate": 750,
  "shift_date":  "2026-07-31",
  "start_time":  "09:00",
}
```

```
"end_time":      "17:00"
}
```

Response — add **branches** to the returned shift (same shape as create-job), and include it wherever this shift is later returned (shift-detail / bookings):

```
{
  "status": true,
  "shift_id": 219,
  "data": [ /* matching candidates */ ],
  "shift": {
    "id": 219,
    "branches": [ {"id":5,"name":"Chicago"}, {"id":6,"name":"New York"} ]
  }
}
```

3 · Search by software / skill in the search bar

Endpoints	GET /recruiter/dashboard (search candidates) · GET /candidate/dashboard (search jobs)
Side	Both
App / screen	Recruiter 'Search Candidates' · Candidate 'Search' (placeholders already say 'software')
Why	Typing a software name (e.g. 'Dentally') should return matching results

Request the app already sends (no frontend change needed):
GET /recruiter/dashboard?search=Dentally&software[]=4&latitude=..&longitude=..&location=..
GET /candidate/dashboard?search=Dentally&software[]=4&latitude=..&longitude=..&location=..

Backend change required: the free-text search parameter must also match the **software name** (join the software table), not only name / profession / title:

```
WHERE (
  name LIKE %search%
OR profession LIKE %search%
OR title LIKE %search%
OR EXISTS (join software s WHERE s.name LIKE %search%)  -- ADD THIS
)
```

The software[] id filter already works structurally — this only fixes the typed-name path. Note: this could not be confirmed live because dashboard filters by radius/location and the test accounts had 0 in-range results. If search already joins software names, item 3 is already done — please confirm.

Summary

All four changes are backend-only. The frontend is already sending and handling everything.

#	Method + URL	Side	Screen	Change
1A	POST /candidate/request-cancellation	candidate→recruiter	Cancellation Requests	Create notification for the recruiter
1B	POST /recruiter/update-cancellation-status	recruiter→candidate	Cancellation Requests	Create notification for the candidate
2	POST /recruiter/search-candidates	recruiter	Fill-In shift form	Accept branch_ids; return branches[{id,name}]
3	GET /recruiter/dashboard & /candidate/dashboard	both	Search screens	search must also match software name